

BDS BOOTCAMP WEEK

Session 1: Getting Set Up

Terminal · Python · VS Code · Environments

WELCOME

How today works

- This is a hands-on session, not a lecture — most of the 2 hours is you doing things on your own laptop
- Goal by the end: everyone has Python, VS Code, and a working environment installed and tested
- Stuck? Just raise a hand — that's exactly what the TAs are here for
- This is session 1 of 3 this week — Git/GitHub and AI/research tools follow

TODAY'S AGENDA

- The terminal, why we use it
- Installing Python
- VS Code tour
- Virtual environments

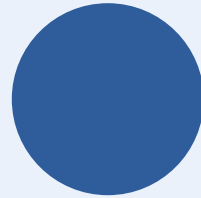
YOUR SUPPORT TEAM

Meet the TAs



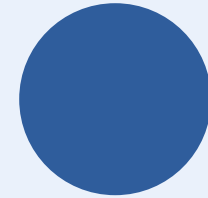
Christian

3rd-semester BDS student
Here to help all week



Andreas

3rd-semester BDS student
Here to help all week



Roger

3rd-semester BDS student
Here to help all week

They've been exactly where you are now — no question is too basic.

PART 1

The Terminal

What is it, and why bother?

- A text-based way to talk to your computer — instead of clicking, you type commands
- Almost every data science tool (Python, Git, Conda) is controlled from here
- It looks intimidating at first — it becomes second nature within a couple of weeks
- Windows: use "Anaconda Prompt" or PowerShell · Mac: use "Terminal"

TRY IT

Open your terminal now
and type:

```
whoami
```

Basic navigation commands

```
pwd / cd
```

Show or change your current folder

```
ls (Mac) / dir (Win)
```

List files in the current folder

```
cd foldername
```

Move into a folder

```
cd ..
```

Move up one folder

```
mkdir name
```

Create a new folder

```
python --version
```

Check if / which Python is installed

PART 2

Installing Python

We'll use Anaconda

- Anaconda = Python + package manager (conda) + lots of data science packages pre-installed
- One installer, works the same way on Windows and Mac — good default for a first setup
- Comes with Anaconda Navigator (GUI) too, but we'll mainly work from the terminal
- Follow the step-by-step install handout for your OS (Windows / Mac) — this is the main hands-on task right now

Verify your installation

Open a terminal and run:

```
conda --version  
python --version
```

If you see version numbers for both — you're set.

If you get "command not found" — flag a TA, this is a normal fix (usually a PATH issue).

PART 3

VS Code

Your main tool for the next 2 years

- A free, lightweight code editor — where you'll write and run almost all your Python code
- Install from code.visualstudio.com
- Install two extensions: "Python" and "Jupyter" (search in the Extensions panel, the puzzle-piece icon)
- You can open a folder as your "workspace" and it remembers your setup
- You may hear about Cursor (a VS Code fork with AI built in) — we're standardizing on plain VS Code so everyone's setup matches; more on AI coding tools in Session 4

A quick tour

- **Explorer** Left sidebar — your files and folders
- **Editor** Center — where you write code
- **Integrated Terminal** Bottom panel — a terminal built right in
- **Extensions** Puzzle icon — install Python & Jupyter here
- **Run ▶ button** Top-right of a script — runs your code

PART 4

Virtual Environments

Why bother?

- Different projects often need different, sometimes conflicting, package versions
- An environment = an isolated "box" of Python + packages just for one project
- Keeps your projects from breaking each other — and makes your work reproducible
- Rule of thumb: one environment per course / per project

Two flavors: conda vs venv

Both solve the same problem — isolating packages per project — with different tools.

conda (we'll use this)

- Comes with Anaconda — already on your laptop from Part 2
- Manages the Python version itself, e.g. `python=3.11`
- Installs packages from conda channels (and can also use pip)
- Commands: `conda create` / `conda activate`

venv (the pip alternative)

- Built into every Python install — nothing extra to install
- Uses whichever Python version is already installed
- Installs packages only via pip
- Commands: `python -m venv` / `activate`

You'll run into venv in tutorials and other courses — good to recognize it, but for this course we'll stick with conda since it manages the Python version and packages together.

Creating your first environment

```
conda create -n bds python=3.11
conda activate bds
conda install pandas jupyter
```

- "bds" is just a name you choose — conda creates and manages this environment for you
- Activate it every time you open a new terminal to work on this course — you'll see (bds) in your prompt once it's active

Kernel not showing up in VS Code?

```
conda activate bds
conda install ipykernel

pip install ipykernel          # fallback if conda install fails (permissions)
python -m ipykernel install --user --name bds
```

- This means ipykernel isn't installed in that conda environment yet — every environment needs its own copy
- conda install can fail with a permissions error on a shared Anaconda install — pip works as a fallback inside the same activated environment
- Then in VS Code: kernel picker → "Select Another Kernel..." → "Jupyter Kernel" → pick it (reload the window if it's still missing)

Creating an environment: the venv way

The pip alternative — built into Python, no extra install needed.

```
python -m venv bds_venv

# Windows:  bds_venv\Scripts\activate
# Mac/Linux: source bds_venv/bin/activate

pip install pandas jupyter
```

- Same idea as conda, different tool — venv uses whichever Python is already installed and pip only
- We're not using this for the course, but good to recognize when you see it elsewhere